



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Laboratorio 4: Desarrollo Frontend con Web Components y Consumo de API REST Transaccional (Continuación del Laboratorio 3)

Metadatos

- **Tiempo Estimado:** 5 horas (trabajo extraclase autónomo)

Herramientas Requeridas:

- Proyecto Java (Spring Boot y SQL Server) desarrollado en el Laboratorio 3.
- Navegador web moderno con herramientas de desarrollo (Chrome/Firefox).
- VS Code con extensión Live Server (o similar).

Metas de Aprendizaje:

1. Configurar el intercambio de recursos de origen cruzado (CORS) en controladores REST de Spring Boot para permitir peticiones web.
2. Diseñar componentes web nativos encapsulados con Shadow DOM (`<template>`, `<slot>`) para desplegar tareas y comentarios en tiempo real.
3. Consumir endpoints transaccionales asíncronos mediante el uso de Fetch API (GET, POST, PUT).
4. Crear estilos dinámicos responsivos utilizando CSS Grid, Nesting y Container Queries.

Introducción

En esta segunda parte de la evaluación práctica, usted transformará el backend desarrollado en el **Laboratorio 3** en una aplicación web interactiva completa (Full-stack). Manteniendo la base de datos GestorTareas[Carné]_II2026 y la arquitectura de capas en Spring Boot, usted diseñará una interfaz de usuario responsiva utilizando **Web Components** nativos del navegador y hojas de estilo CSS modernas.

El frontend se comunicará directamente con los endpoints del backend para listar tareas y comentarios, agregar nuevos registros transaccionales y actualizar estados aprovechando las ventajas de optimización (como JOIN FETCH y *Dirty Checking*) implementadas previamente.

Parte 1: Estructura del Proyecto

Cree una carpeta llamada `frontend` al mismo nivel de su proyecto backend del Laboratorio 3 para estructurar sus componentes:

```
Laboratorio_4/  
  ■■■ backend/           <-- Proyecto Spring Boot (Laboratorio 3)  
  ■ ■■■ src/...  
  ■■■ frontend/         <-- Capa Frontend (Nuevo entregable)  
    ■■■ index.html       <-- Portal General  
    ■■■ style.css        <-- Estilos Globales y Grid Elástico  
    ■■■ components/
```

```
■■■ TarjetaTarea.js    <-- Web Component: Tarjeta de Tarea
■■■ PanelTareas.js    <-- Web Component: Formulario y Listado
```

Parte 2: Modificaciones en el Backend (CORS)

Para permitir que el navegador web del cliente cargue la información desde un servidor local de frontend (ej. ejecutándose en el puerto 5500) hacia el puerto del backend (8080), debe habilitar la seguridad CORS.

1. Abra el controlador `TareaController.java` desarrollado en el Laboratorio 3.
2. Decore la clase del controlador con la anotación `@CrossOrigin` para habilitar las peticiones asíncronas de origen cruzado:

```
@RestController
@RequestMapping("/api/tareas")
@CrossOrigin(origins = "*") // Habilitar peticiones desde cualquier origen
public class TareaController {
    // ...
}
```

Parte 3: Especificaciones del Frontend (Web Components)

La interfaz se estructurará con dos componentes web nativos:

1. Elemento Tarjeta de Tarea (`TarjetaTarea.js`)

Debe completar el componente `<tarjeta-tarea>` para renderizar cada tarea individual de forma aislada: * **Shadow DOM:** Implemente Shadow DOM en modo abierto (open) para aislar los estilos de la tarjeta. * **Marcado y Slots:** Utilice una etiqueta `<template>` para definir la estructura HTML del componente. Utilice `<slot>` para colocar el título de la tarea, la descripción, el colaborador asignado y las reseñas o comentarios dinámicamente. * **Comentarios Embebidos:** Renderice la lista de comentarios asociados recorriendo la lista devuelta por el JSON de la tarea. * **Estilos y Container Queries:** Agregue estilos CSS específicos dentro del Shadow DOM. Si el contenedor de la tarjeta

medida menos de 400px de ancho, los elementos deben ordenarse verticalmente con colores de fondo sutiles basados en el estado (Rojo para "Pendiente", Amarillo para "En Progreso", Verde para "Completada").

2. Elemento Panel de Tareas (`PanelTareas.js`)

Debe crear el componente contenedor `<panel-tareas>` que controle el estado general de la aplicación:

- * **Renderizado de Formulario y Grid:** Incluya un formulario para registrar nuevas tareas (debe solicitar título, descripción, estado e ID del colaborador) y una retícula (`.tareas-grid`) para el catálogo.
- * **Petición GET (Optimizado):** Realice una petición `fetch` al endpoint `/api/tareas/optimizada` implementado en el Laboratorio 3. Recorra la lista del JSON e instancie elementos `<tarjeta-tarea>`, asignándole atributos `data-` y agregándolos a la retícula.
- * **Petición POST (Registro):** Capture el evento submit del formulario, valide que los campos requeridos no estén en blanco y envíe los datos estructurados en formato JSON al endpoint `POST /api/tareas`. Si responde de forma exitosa (201 Created), limpie los campos y recargue la lista de tareas en pantalla sin refrescar la página.
- * **Petición PUT (Actualización transaccional):** Incorpore un botón de "Completar" en cada tarjeta que invoque el endpoint `PUT /api/tareas/{id}/completar`. La lista debe refrescarse al completarse el proceso.

3. Hoja de Estilos Global (`style.css`)

Diseñe un portal de apariencia moderna utilizando selectores avanzados de CSS:

- * **CSS Nesting:** Anide las reglas del formulario, inputs y estados del botón de registrar, evitando repetir selectores base.
- * **Grid Elástico:** Configure la clase `.tareas-grid` con `display: grid` y auto-ajuste de columnas responsivo (`repeat(auto-fit, minmax(300px, 1fr))`), logrando adaptabilidad a pantallas móviles y de escritorio sin usar `@media`.

Pistas y Ayudas (Hints para el Éxito)

Pista 1: Cabeceras JSON de Fetch

Al realizar peticiones de tipo `POST` o `PUT` enviando cuerpos JSON en Javascript, recuerde configurar obligatoriamente los encabezados correspondientes en el método `fetch` para que Spring Boot logre mapear la entrada:

```
headers: {  
  'Content-Type': 'application/json'  
}
```

Pista 2: Evitar Conflictos de Shadow DOM

Recuerde que el estilo dentro de un Shadow DOM no se ve afectado por la hoja de estilos global (`style.css`). Si desea usar variables de color o tipografías de Google Fonts dentro del componente `<tarjeta-tarea>`, defina las fuentes en la etiqueta `@import` al inicio del bloque `<style>` interno del componente o consuma propiedades personalizadas de CSS (`--mi-variable`).

Pista 3: Formatear Fechas en Javascript

Las fechas devueltas en el JSON por Spring Data Auditing vienen en formato de marca de tiempo ISO (ej. `2026-08-24T11:00:00`). Usted puede convertirlas a formato local en español para mejorar la experiencia de usuario:

```
const fechaLocal = new Date(fechaISO).toLocaleString('es-CR');
```

Rúbrica de Evaluación

La calificación definitiva de este Laboratorio 4 se obtendrá bajo los siguientes criterios de evaluación:

Criterio	Porcentaje	Descripción Detallada
Backend REST y Habilitación de CORS	15%	Anotación <code>@CrossOrigin</code> implementada en controladores y verificación de peticiones exitosas sin bloqueos de red en el navegador.

Componente Tarjeta de Tarea	25%	Creación de <code><tarjeta-tarea></code> con Shadow DOM abierto, uso correcto de slots para inyectar datos y despliegue del listado de comentarios.
Contenedor Panel de Tareas	25%	Creación de <code><panel-tareas></code> , integración del formulario y renderizado dinámico de tarjetas iterando el JSON del API REST.
Peticiones Fetch (POST, PUT, GET)	20%	Consumo asíncrono asertivo de endpoints de lectura optimizada (JOIN <code>FETCH</code>), inserción de tareas e invocación del Dirty Checking (PUT).
Diseño CSS Moderno	15%	Diseño responsivo elástico usando CSS Grid, selectores anidados (CSS Nesting) y Container Queries según el estado de la tarea.
Total	100%	Nota final del Laboratorio 4